Rúbrica de Evaluación – TPI Programación 1

Sistema de Control de Inventario en Python: lista de diccionarios, funciones y manejo de errores

La corrección se hace en **tres instancias, siempre en este orden** (enunciado, sección 9). Cada defecto se juzga en una sola instancia: la primera que lo alcanza.

Instancia 1 – Condiciones de entrega (antes de puntuar)

Si no se cumple alguna, **el trabajo no se evalúa**.

- **ZIP** con nombre **Apellido_Nombre_P1.zip**, que contenga **todos los archivos .py** y el **informe en PDF**.
- **Video** de **entre 5 y 10 minutos**, con el link (YouTube o Google Drive) en el comentario de la entrega y **permisos de acceso para el docente**.
- Se respetan las restricciones de entrega (7.4): no se califican archivos subidos en comentarios, fuera de la plataforma institucional ni código enviado por enlace a repositorios.

La instancia 1 solo controla que el PDF esté. Si al informe le falta el diagrama u otro componente, se considera en el criterio del informe (Instancia 3).

Instancia 2 – Reglas técnicas (tope de calificación)

El incumplimiento de cualquiera de las reglas 2.1 a 2.6 **limita la calificación a un máximo de 10 sobre 100 (10%)**. Es un tope, no un descuento: la Instancia 3 se aplica igual y el resultado no puede superar 10.

Regla	Se incumple cuando…	Cumple	No cumple
2.1 Estructuras permitidas	Se usan estructuras de control u organización fuera de las permitidas (secuencia, if/elif/else, while/for con break/continue, match/case para el menú, def/return, try/except/raise). No cuentan como estructuras, y están permitidos: **input**, **print**, **int**, **len**, **range**, los f-strings, **strip()/lower()** y dividir el programa en varios archivos **.py** propios (ver 2.1).	☐	☐
2.2 Estructura de datos	El inventario no es una lista de diccionarios llamada **inventario**, con exactamente las claves **'herramienta'** y **'cantidad'**.	☐	☐
2.3 Modularización	Alguna de las opciones 1 a 6 no tiene una función propia, o el programa está todo en el bloque principal.	☐	☐
2.4 Manejo de errores	El programa no usa **try/except**, o no usa **raise**.	☐	☐
2.5 Prohibiciones	Se usan clases (incluida una clase propia de excepción, como class MiError(Exception)), diccionarios sueltos (fuera de inventario.append(...)) u otras colecciones de datos fuera de inventario.	☐	☐
2.6 Variables globales	Se usa alguna variable global (inventario debe crearse dentro de una función y pasarse por parámetro).	☐	☐

Una validación puntual que falta o está mal resuelta no activa el tope: reduce el nivel de logro del criterio de validaciones.

Instancia 3 – Rúbrica con puntajes por nivel

Puntaje del criterio = puntaje máximo × porcentaje del nivel alcanzado. Total: **100 puntos**.

I. Funcionamiento y Desarrollo del Software (60%)

Criterio	Excelente (100%)	Bueno (75%)	Satisfactorio (50%)	Insuficiente (25%)	No presentado (0%)
1. Funcionamiento de las siete opciones (25)	Las siete opciones existen y hacen todo lo que pide la sección 4: carga múltiple (una segunda ejecución conserva lo ya cargado), listado, consulta, reporte de agotados (con un mensaje propio cuando ninguna herramienta tiene stock cero), alta, venta/ingreso en el orden de pasos indicado y salida. El menú es persistente y solo la opción 7 termina el programa.	Las siete opciones funcionan, pero una sola omite un detalle de la sección 4 (p. ej., el reporte no distingue el caso sin agotados, o la opción 6 no sigue el orden de pasos indicado).	Las siete opciones existen y se pueden usar, pero dos o más omiten parte de lo que pide la sección 4.	Falta alguna opción, o alguna no se puede usar (p. ej., no se pueden cargar herramientas).	No aplica.
2. Validaciones y recuperación de errores (25)	Todas las situaciones de la sección 5 se resuelven con el mecanismo exigido: **raise ValueError** para reglas de negocio (nombre vacío/duplicado, stock negativo, cantidades ≤ 0, venta mayor al stock, opción fuera de rango), **try/except** para conversiones, y condición + mensaje, sin raise, para casos informativos (inventario vacío, herramienta inexistente, tipo de movimiento inválido). La opción 1 reintenta dentro de la función (conserva el nombre válido); las opciones 3, 5 y 6 vuelven al menú vía main(). Nombres comparados sin mayúsculas ni espacios extremos.	Una sola situación de la sección 5 no se resuelve como indican sus tablas (p. ej., no ignora mayúsculas al detectar duplicados), sin dejar datos a medias ni modificar el stock.	Dos o más situaciones de la sección 5 no se resuelven como indican sus tablas, pero ningún dato inválido deja datos a medias ni modifica el stock.	Un dato inválido deja datos a medias o modifica el stock, o hace que el programa se interrumpa.	No aplica.
3. Organización y legibilidad del código (10)	Cada función hace una tarea clara y todo el código usa **snake_case**.	Una sola función mezcla varias tareas, o algún nombre aislado no usa **snake_case**.	Varias funciones mezclan tareas, o varios nombres no usan **snake_case**.	No se distingue qué hace cada función, o el código es difícil de leer.	No aplica.

II. Informe de Diseño y Fundamentación (25%)

Criterio	Excelente (100%)	Bueno (75%)	Satisfactorio (50%)	Insuficiente (25%)	No presentado (0%)

Criterio	Excelente (100%)	Bueno (75%)	Satisfactorio (50%)	Insuficiente (25%)	No presentado (0%)
1. Conceptos aplicados (12)	Explica listas, diccionarios, funciones, condicionales, bucles y try/except/raise, **vinculando cada concepto con una parte concreta** de su programa.	Explica todos los conceptos y la mayoría vinculados con el programa, con alguna omisión menor.	Explica el diseño concreto pero omite algún concepto, o las vinculaciones son escasas.	Solo reproduce definiciones generales y no permite comprender la solución presentada.	No presenta.
2. Diagrama de flujo (8)	Diagrama propio y claro del menú y de los recorridos de carga, validación de datos y actualización de stock, coherente con el código.	Diagrama correcto pero omite alguno de los recorridos o tiene pequeñas incoherencias con el código.	Diagrama parcial o genérico (p. ej., copia el del enunciado sin reflejar su diseño).	Diagrama confuso o sin relación con el programa.	No presenta.
3. Lenguaje, entorno y recursos (5)	Un párrafo por cada recurso (qué es, versión usada y por qué lo eligió). Si hay varios .py, indica cuál se ejecuta para iniciar.	Menciona los recursos con alguna omisión (versión o justificación).	Solo enumera los recursos, sin investigación ni justificación.	Información incorrecta o mínima.	No presenta.

III. Video Explicativo (15%)

Criterio	Excelente (100%)	Bueno (75%)	Satisfactorio (50%)	Insuficiente (25%)	No presentado (0%)
1. Explicación (8)	Explica, sin leer el código, la estructura, el rol de cada función, por qué eligió las estructuras de control y el diseño modular, y cómo aplica try/except y raise.	Explica con claridad, pero omite o trata superficialmente alguno de esos puntos.	Omite varios puntos o la explicación es superficial.	Lee el código sin explicarlo.	No aplica.
2. Demostración y condiciones de grabación (7)	Sigue la secuencia del menú de la sección 8: éxito en cada opción y un caso de error en las opciones 1, 3, 5 y 6. En la opción 1 carga una herramienta con stock positivo y otra con stock 0; en la 3 muestra además una herramienta inexistente; en la 6 muestra una venta y un ingreso con éxito. Se muestra a cámara, con el código en VS Code, fondo claro y letra grande.	Demostración casi completa: falta uno de esos casos o una condición de grabación.	La demostración queda incompleta: faltan varias opciones o varios de esos casos.	No muestra el funcionamiento del programa.	No aplica.

Escala de puntuación

- **90–100 puntos (Sobresaliente):** Proyecto supera expectativas, implementación robusta y bien documentada.
- **70–89 puntos (Notable):** Cumple la mayoría de los criterios con buena calidad.
- **50–69 puntos (Aprobado):** Cumple los mínimos requeridos, aunque con deficiencias claras.

- **0–49 puntos (Reprobado):** No alcanza los mínimos, implementación incompleta o incorrecta.

Nota sobre 100. Si se incumple alguna regla técnica (Instancia 2), la nota final no puede superar 10.

Planilla de corrección

Estudiante: _____ **Comisión:** _____ **Fecha:** ____/____/_____ **Corrector/a:** _____

Instancia 1 – Entrega: ☐ Cumple ☐ No se evalúa **Instancia 2 – Reglas técnicas:** ☐ Cumple todas ☐ Aplica tope 10 (reglas: _____)

Criterio	Máx.	Nivel (100/75/50/25/0)	Puntaje	Observaciones
I.1 Funcionamiento de las siete opciones	25			
I.2 Validaciones y recuperación de errores	25			
I.3 Organización y legibilidad del código	10			
II.1 Conceptos aplicados	12			
II.2 Diagrama de flujo	8			
II.3 Lenguaje, entorno y recursos	5			
III.1 Video – Explicación	8			
III.2 Video – Demostración	7			
TOTAL	**100**			**Nota final (/100):**

Casos clave para verificar en la corrección

Selección de las pruebas manuales del enunciado (sección 10) que distinguen los niveles de los criterios I.1 y I.2.

Prueba	Estado inicial	Resultado esperado	OK
Menú: ingresar abc / elegir 9	Cualquiera	Muestra el error (de conversión / de rango) y vuelve a pedir opción.	☐
Op. 1: cantidad 0, -1, abc o 3.5	Inventario vacío	Informa el error y vuelve a pedir ese dato.	☐
Op. 1: cargar martillo / ␣Martillo␣	Martillo cargado	Rechaza como duplicado y vuelve a pedir el nombre.	☐
Op. 1: Pala con stock -2 o abc	Martillo cargado	Vuelve a pedir solo el stock, conservando Pala; no se agrega hasta ser válido.	☐
Op. 1: Pala con stock 0	Martillo cargado	Se acepta y aparece en el reporte de agotados.	☐
Op. 2, 3, 4, 6 sin datos	Inventario vacío	Informa que no hay herramientas cargadas, sin pedir datos.	☐
Op. 3: consultar ␣MARTILLO␣	Martillo con 10	Encuentra la herramienta y muestra 10.	☐
Op. 4: reportar agotados	Todas con stock positivo	Informa que ninguna tiene stock cero (mensaje distinto al de vacío).	☐
Op. 5: alta de Martillo	Martillo cargado	Rechaza como duplicado sin pedir stock y vuelve al menú.	☐

Prueba	Estado inicial	Resultado esperado	OK
Op. 5: alta de Pala con stock 3.5	Martillo cargado	Muestra el motivo y vuelve al menú; no queda registro incompleto.	☐
Op. 6: actualizar Pala (inexistente)	Martillo cargado	Informa que no existe; no pide unidades.	☐
Op. 6: tipo de movimiento inválido	Martillo con 10	Lo rechaza con mensaje y no modifica el inventario.	☐
Op. 6: unidades 0, -1, abc o 3.5	Martillo con 10	Rechaza el movimiento; el stock sigue en 10.	☐
Op. 6: vender 4 unidades	Martillo con 3	Informa el error; el stock sigue en 3.	☐
Op. 6: ingresar 2 unidades	Martillo con 0	Stock queda en 2 y deja de aparecer en agotados.	☐